Fundamentals of AI and KR - Module 3

4. Exact inference

Roberta Calegari

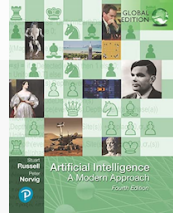
Fall 2026

Exact inference

From representing uncertainty to answering queries

A Bayesian network compactly represents a joint probability distribution.

We now study how to compute posterior probabilities directly from its local conditional distributions.



Inference tasks I

What can we ask using the alarm network?

Suppose John and Mary report that the alarm is sounding. What can the model help us determine?

- Marginal query: how likely is a burglary?

$$P(Burglary \mid JohnCalls = true,\ MaryCalls = true).$$

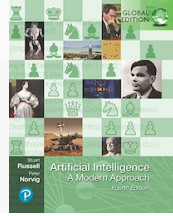
- Joint query: how likely is each combination of burglary and earthquake?

$$P(Burglary, Earthquake \mid JohnCalls = true,\ MaryCalls = true).$$

- Explanation: which underlying events best explain the observed calls?

Connection

Posterior probabilities help us compare possible explanations of the observations.



Inference tasks II

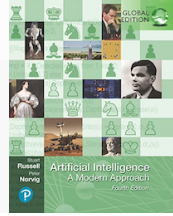
Further applications of probabilistic inference

Probability calculations also support decisions and the analysis of the model itself.

- Optimal decisions: *should we send someone to inspect the house?*
  Probabilities of relevant events + costs and benefits of each action.
- Value of information: *is it worth checking whether an earthquake was reported before deciding?*
  Consider additional evidence to justify its cost.
- Sensitivity analysis: *how much would our conclusion change if the alarm's false positive rate were different?*

Our focus

Probability calculations underlying these tasks: answering probability queries.



Inference tasks III

Probability query

A probability query asks for the conditional distribution of query variables $\mathbf{Y}$, given observed evidence $\mathbf{E} = \mathbf{e}$:

$$\mathbf{P}(\mathbf{Y} \mid \mathbf{E} = \mathbf{e}).$$

- Marginal query: one query variable.

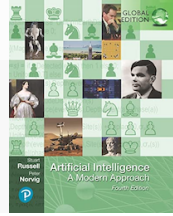
$$\mathbf{P}(X_i \mid \mathbf{E} = \mathbf{e}).$$

- Joint query: several query variables.

$$\mathbf{P}(X_i, X_j \mid \mathbf{E} = \mathbf{e}).$$

Key question

How can we compute these distributions directly from the local probability models of a Bayesian network?



Inference by enumeration: the query

The observed evidence

John and Mary both call. How likely is a burglary?

$$P(B \mid j, m).$$

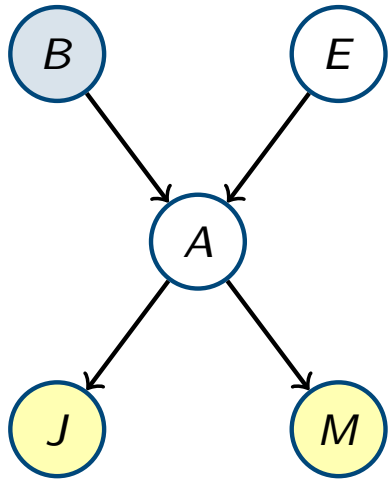
Our variables are:

- query variable: B;
- evidence: $J = true$, $M = true$;
- hidden variables: E, A.

We must consider both possible values of B:

$$P(b \mid j, m), \qquad P(\neg b \mid j, m).$$

Here, b means $B = true$, and similarly for e, a, j, m.



Yellow: observed variables

From a posterior to two joint probabilities

By the definition of conditional probability:

$$P(b \mid j, m) = \frac{P(b, j, m)}{P(j, m)}.$$

The denominator is shared by both values of B:

$$P(j, m) = P(b, j, m) + P(\neg b, j, m).$$

We therefore need a way to compute $P(b, j, m)$ and $P(\neg b, j, m)$ from the network.

Enumerate the hidden possibilities

First, fix $B = \mathit{true}$, together with the evidence j, m.
We do not know whether an earthquake occurred or the alarm sounded. There are four possible assignments to E, A:

E	A	**Corresponding joint probability**
true	*true*	$P(b, e, a, j, m)$
true	*false*	$P(b, e, \neg a, j, m)$
false	*true*	$P(b, \neg e, a, j, m)$
false	*false*	$P(b, \neg e, \neg a, j, m)$

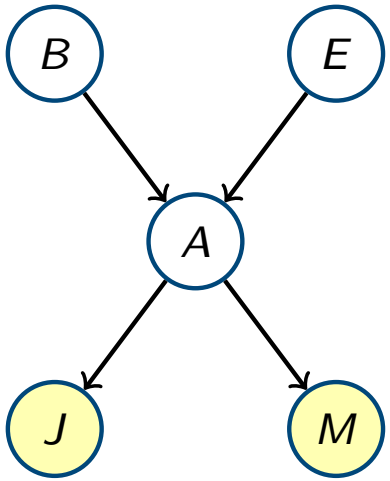
Sum the mutually exclusive possibilities

$$P(b, j, m) = P(b, e, a, j, m) + P(b, e, \neg a, j, m) + P(b, \neg e, a, j, m) + P(b, \neg e, \neg a, j, m).$$

Repeat the same calculation with $B = \mathit{false}$.

Read probabilities from the network's CPTs

For every complete assignment, select one probability from each local distribution.



$J = true,\ M = true$
observed

$$P(b) = 0.001, \qquad P(e) = 0.002.$$

B	E	$P(a \mid B, E)$	$P(\neg a \mid B, E)$
true	*true*	0.95	0.05
true	*false*	0.94	0.06
false	*true*	0.29	0.71
false	*false*	0.001	0.999

A	$P(j \mid A)$	$P(m \mid A)$
true	0.90	0.70
false	0.05	0.01

Network factorization

$$P(B, E, A, J, M) = P(B)\,P(E)\,P(A \mid B, E) \cdot P(J \mid A)\,P(M \mid A).$$

One complete assignment: select and multiply

Consider the world: $B = true, \quad E = true, \quad A = true, \quad J = true, \quad M = true.$

Node	Selected CPT entry	Value
B	$P(b)$	0.001
E	$P(e)$	0.002
A	$P(a \mid b, e)$	0.95
J	$P(j \mid a)$	0.90
M	$P(m \mid a)$	0.70

$$\begin{aligned} P(b, e, a, j, m) &= P(b)P(e)P(a \mid b, e)P(j \mid a)P(m \mid a) \\ &= 0.001 \cdot 0.002 \cdot 0.95 \cdot 0.90 \cdot 0.70 = 0.000001197. \end{aligned}$$

$$\begin{aligned} P(b, j, m) = \underline{\mathbf{P(b,e,a,j,m)}} &+ P(b, e, \neg a, j, m) \\ &+ P(b, \neg e, a, j, m) + P(b, \neg e, \neg a, j, m). \end{aligned}$$

Repeat for every hidden assignment, then sum

Keep $B = \mathit{true}$ and the evidence j, m fixed. For each assignment of E, A, select the corresponding CPT entries:

E	A	**Product of selected entries**
true	*true*	$P(b)P(e)P(a \mid b, e)P(j \mid a)P(m \mid a)$
true	*false*	$P(b)P(e)P(\neg a \mid b, e)P(j \mid \neg a)P(m \mid \neg a)$
false	*true*	$P(b)P(\neg e)P(a \mid b, \neg e)P(j \mid a)P(m \mid a)$
false	*false*	$P(b)P(\neg e)P(\neg a \mid b, \neg e)P(j \mid \neg a)P(m \mid \neg a)$

Sum the four products

Their sum is the unnormalized weight: $q_b = P(b, j, m)$.

Then repeat with $B = \mathit{false}$

Compute four more products to obtain $q_{\neg b} = P(\neg b, j, m)$. Finally, normalize the two weights to obtain $P(B \mid j, m)$.

Inference by enumeration: the procedure

Inference by enumeration

Answer a probability query by summing over assignments of hidden variables, computing each joint probability from the network's local conditional distributions.

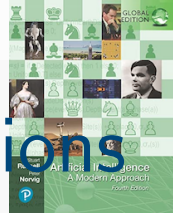
For our query:

1. Fix the evidence j, m.
2. For each value of B, enumerate the assignments of E, A.
3. Multiply the corresponding CPT entries and sum to obtain q_b and $q_{\neg b}$.
4. Normalize:

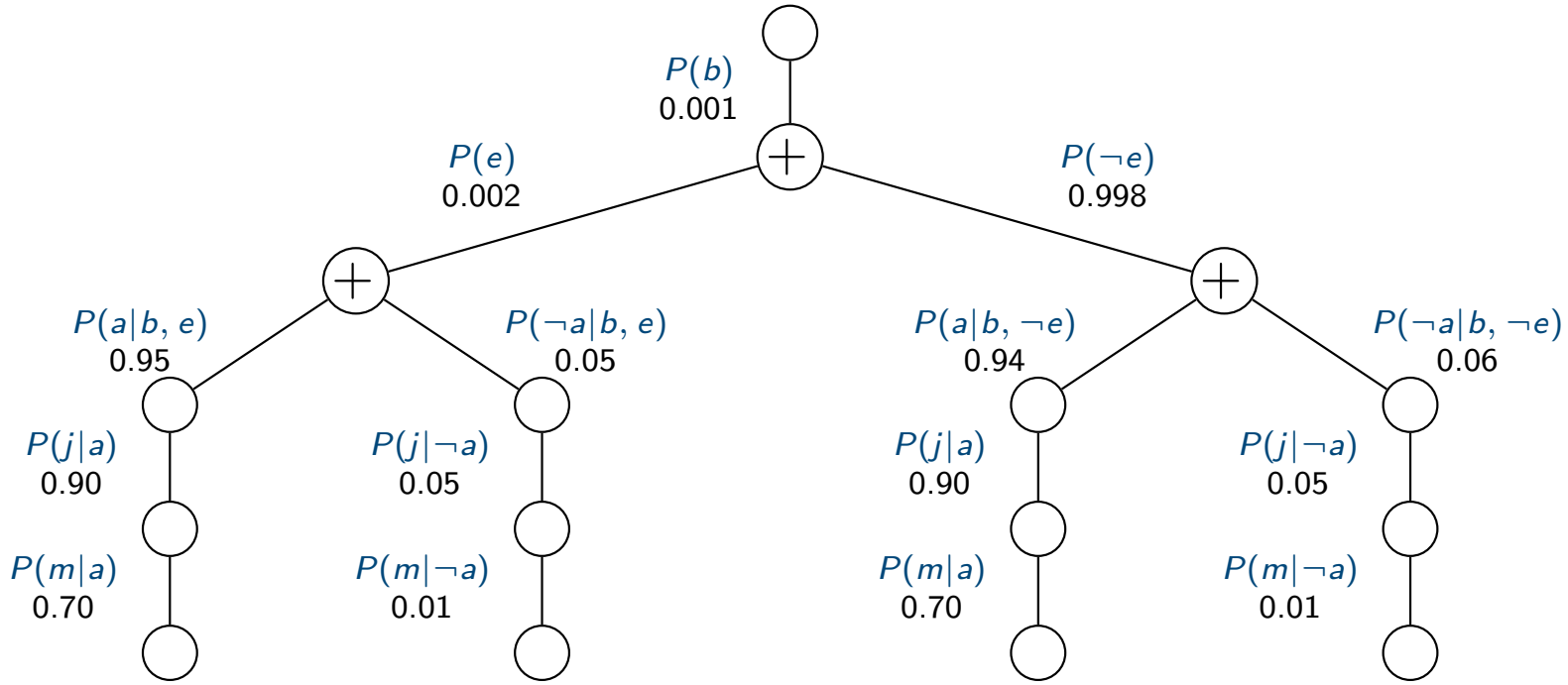
$$P(B \mid j, m) = \alpha \langle q_b, q_{\neg b} \rangle, \qquad \alpha = \frac{1}{q_b + q_{\neg b}}.$$

What have we gained?

We compute the required sums directly from the CPTs, without storing the full joint probability table.



Evaluation tree: enumeration repeats the same calculations



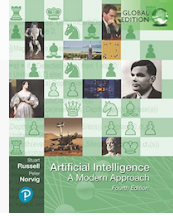
Recursive depth-first enumeration

$O(n)$ space (no need to construct full joint), $O(d^n)$ time

Inefficient: Same products, as $P(j \mid a)P(m \mid a)$, evaluated in multiple branches.

Can we do better?

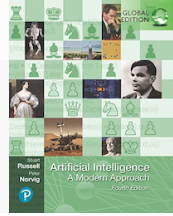
Compute repeated expressions once, store the results, and reuse them.



Enumeration algorithm

function ENUMERATION-ASK(X,**e**,bn) **returns** a distribution over X
  inputs: X, the query variable
    e, observed values for variables **E**
    bn, a Bayesian network with variables $\text{VARS}[bn] = \{X\} \cup \mathbf{E} \cup \mathbf{Y}$
  $\mathbf{Q}(X) \leftarrow$ a distribution over X, initially empty
  for each value x_i of X **do**
    extend **e** with $X = x_i$
    $\mathbf{Q}(x_i) \leftarrow$ ENUMERATE-ALL(VARS[bn],**e**)
  return NORMALIZE($\mathbf{Q}(X)$)

function ENUMERATE-ALL($vars$,**e**) **returns** a real number
  if EMPTY?($vars$) **then return** 1.0
  $Y \leftarrow$ FIRST($vars$)
  if Y has value y in **e** **then**
    return $P(y|Parents(Y)) \times$ ENUMERATE-ALL(REST($vars$),**e**)
  else return $\sum_y P(y|Parents(Y)) \times$ ENUMERATE-ALL(REST($vars$),**e** $\cup \{Y = y\}$)



From enumeration to variable elimination

We still want to compute:

$$P(B \mid j, m).$$

Enumeration evaluates products for every hidden assignment. Some expressions are repeated, for example:

$$P(j \mid a)P(m \mid a).$$

Can we reuse this calculation?

This product depends only on A. Compute its value once for each value of A, store the results, and reuse them for different values of B, E.

Variable elimination

An exact inference algorithm that sums out hidden variables one at a time, storing intermediate results as factors to avoid repeated computation.

What is a factor?

Factor

A factor is a nonnegative function of a set of variables. For discrete variables, it can be represented as a table with one value for each assignment.
Its scope is the set of variables on which it depends.

A factor obtained by fixing evidence

The CPT $P(J \mid A)$ involves J, A.
After observing $J = \mathit{true}$, we obtain a factor over A:

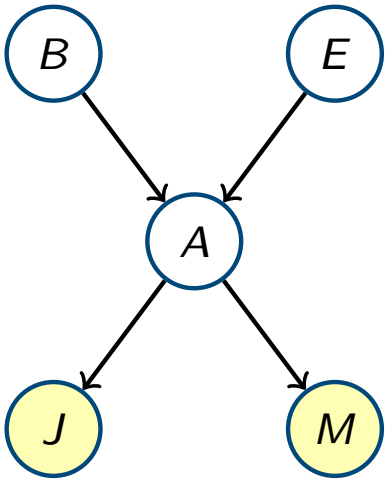
$$f_J(A) = P(j \mid A).$$

A	$f_J(A)$
true	0.90
false	0.05

A factor need not sum to 1: here, $0.90 + 0.05 = 0.95$.

Step 1: one initial factor for each local model

Each node has a local probability model: these 5 models are the starting factors.



Node	Local probability model
B	$P(B)$
E	$P(E)$
A	$P(A \mid B, E)$
J	$P(J \mid A)$
M	$P(M \mid A)$

Why can we treat them as factors?

Each local model is a table of nonnegative values. Multiplying the five tables gives the joint distribution:

$$P(B, E, A, J, M) = P(B)P(E)P(A \mid B, E) \cdot P(J \mid A)P(M \mid A).$$

Step 1: fix the observed values I

Our evidence is $J = true$, $M = true$. We keep only the entries corresponding to those observed values.

John's CPT becomes a factor over A

$$P(J \mid A) \xrightarrow{J=true} f_J(A) = P(j \mid A).$$

A	$f_J(A)$
true	0.90
false	0.05

Original CPT

A	J	$P(J \mid A)$
true	*true*	0.90
true	*false*	0.10
false	*true*	0.05
false	*false*	0.95

$\longrightarrow$

Resulting factor

A	$f_J(A) = P(j \mid A)$
true	0.90
false	0.05

Step 1: fix the observed values II

We also observe $M = true$. Select the entries where $M = true$.

Original CPT

A	M	$P(M \mid A)$
true	*true*	0.70
true	*false*	0.30
false	*true*	0.01
false	*false*	0.99

$\longrightarrow$

Resulting factor

A	$f_M(A) = P(m \mid A)$
true	0.70
false	0.01

Step 1: fix the observed values III

The five initial factors after fixing the evidence

Node	Original local model	Initial factor
B	$P(B)$	$f_B(B) = P(B)$
E	$P(E)$	$f_E(E) = P(E)$
A	$P(A \mid B, E)$	$f_A(A, B, E) = P(A \mid B, E)$
J	$P(J \mid A)$	$f_J(A) = P(J = true \mid A)$
M	$P(M \mid A)$	$f_M(A) = P(M = true \mid A)$

Only the highlighted models are restricted by the evidence: J, M are fixed, while A remains variable.

Step 2: express the calculation using factors

For each query value $b' \in \{true, false\}$, first compute an unnormalized weight: $q(b') = P(B = b', j, m)$ summing over the hidden variables E, A:

$$q(b') = \sum_E \sum_A f_B(b') f_E(E) f_A(A, b', E) f_J(A) f_M(A).$$

Reorganize the same sum

Factors that do not depend on a summed variable can be moved outside:

$$q(b') = f_B(b') \sum_E f_E(E) \left[\sum_A f_A(A, b', E) f_J(A) f_M(A) \right].$$

We will evaluate the inner sum over A first, then the sum over E.

Finally, normalize:

$$P(B = b' \mid j, m) = \frac{q(b')}{q(true) + q(false)} = \alpha q(b').$$

Step 2 operation 1: multiply factors

Pointwise product

Multiply factor entries that agree on the values of their shared variables.
The resulting factor has the union of their scopes.

For example:

$$g(A) = f_J(A) f_M(A).$$

A	$f_J(A)$	$f_M(A)$	$g(A)$
true	0.90	0.70	0.63
false	0.05	0.01	0.0005

Why is this useful?

We can reuse these two products for every assignment of B, E.

More generally, multiplying a factor over A, B and one over B, E produces a factor over A, B, E.

Step 2 operation 2: sum out A I

We want to eliminate the hidden variable A, while keeping B, E. The factors involving A are:

$$f_A(A, B, E) = P(A \mid B, E), \qquad g(A) = P(j \mid A)P(m \mid A).$$

From the alarm's original CPT

B	E	$P(a \mid B, E)$	$P(\neg a \mid B, E)$
true	*true*	0.95	0.05
true	*false*	0.94	0.06
false	*true*	0.29	0.71
false	*false*	0.001	0.999

Already computed

A	$g(A)$
true	0.63
false	0.0005

For each fixed pair B, E, multiply the matching entries for $A = \mathit{true}$ and $A = \mathit{false}$, then add the two products.

Step 2 operation 2: sum out A II

Fix $B = true, E = true$. The highlighted CPT row gives:

$$P(a \mid b, e) = 0.95, \qquad P(\neg a \mid b, e) = 0.05.$$

We must account for both possible values of the hidden variable A:

A	$P(A \mid b, e)$	$g(A)$	**Product**
true	0.95	0.63	0.5985
false	0.05	0.0005	0.000025

Add the two contributions

$$P(j, m \mid B = true, E = true) = 0.95 \cdot 0.63 + 0.05 \cdot 0.0005$$
$$= 0.5985 + 0.000025 = \boxed{0.598525}.$$

It combines both possibilities for A: no longer need to keep separate entries.

Step 2 Sum out A: repeat for every pair B, E

Summing out a variable

For each assignment of the remaining variables, sum over all values of the variable being eliminated:

$$h(B, E) = \sum_A f_A(A, B, E) g(A).$$

Here:

$$h(B, E) = P(a \mid B, E) \cdot 0.63 + P(\neg a \mid B, E) \cdot 0.0005.$$

B	E	**Calculation**	$h(B, E)$
true	*true*	$0.95 \cdot 0.63 + 0.05 \cdot 0.0005$	0.598525
true	*false*	$0.94 \cdot 0.63 + 0.06 \cdot 0.0005$	0.592230
false	*true*	$0.29 \cdot 0.63 + 0.71 \cdot 0.0005$	0.183055
false	*false*	$0.001 \cdot 0.63 + 0.999 \cdot 0.0005$	0.0011295

New factor depends only on B, E: both values of A included in each entry.

Step 3: sum out E

After eliminating A, the query is:

$$P(B \mid j, m) = \alpha f_B(B) \sum_E f_E(E) h(B, E).$$

Define the next intermediate factor:

$$k(B) = \sum_E f_E(E) h(B, E).$$

Using $P(e) = 0.002$ and $P(\neg e) = 0.998$:

$$k(\mathit{true}) = 0.002 \cdot 0.598525 + 0.998 \cdot 0.592230 = 0.59224259,$$

$$k(\mathit{false}) = 0.002 \cdot 0.183055 + 0.998 \cdot 0.0011295 = 0.001493351.$$

The hidden variables A, E have both been eliminated. Only factors over the query variable B remain:

$$P(B \mid j, m) = \alpha f_B(B) k(B).$$

Step 4: multiply the remaining factors and normalize

Multiply by the prior $f_B(B) = P(B)$:

$$\begin{aligned} q_b &= 0.001 \cdot 0.59224259 \\ &= 0.00059224259, \end{aligned}$$

$$\begin{aligned} q_{\neg b} &= 0.999 \cdot 0.001493351 \\ &= 0.001491857649. \end{aligned}$$

Normalize the two weights:

$$P(B \mid j, m) = \frac{\langle q_b, q_{\neg b} \rangle}{q_b + q_{\neg b}} \approx \langle 0.2842,\ 0.7158 \rangle.$$

Interpretation

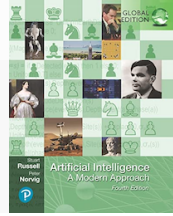
Given that both John and Mary called, the posterior probability of a burglary is approximately 28.4%.

Enumeration and variable elimination give the same result. Variable elimination reorganizes the calculation to reuse intermediate results.

Variable elimination: the general procedure

1. Fix the evidence: restrict the local distributions to the observed values, obtaining initial factors.
2. Choose an order for eliminating hidden variables.
3. For each hidden variable Z:
   - collect all current factors involving Z;
   - multiply them and sum out Z;
   - replace the collected factors with the resulting factor.
4. Multiply the remaining factors.
5. Normalize to obtain the posterior distribution over the query variables.

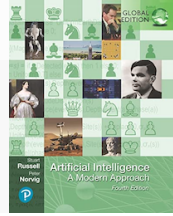
Eliminate a variable only after combining all factors that currently depend on it. Factors that do not involve that variable remain unchanged.



Variable elimination algorithm

function ELIMINATION-ASK(X,**e**,bn) **returns** a distribution over X
  inputs: X, the query variable
    e, evidence specified as an event
    bn, a Bayesian network specifying joint distribution $\mathbf{P}(X_1, \ldots, X_n)$
  $factors \leftarrow [\,]$
  for each var **in** ORDER(VARS[bn]) **do**
    $factors \leftarrow [\text{MAKE-FACTOR}(var, \mathbf{e}) | factors]$
    if var is a hidden variable **then**
      $factors \leftarrow \text{SUM-OUT}(var, factors)$
  return NORMALIZE(POINTWISE-PRODUCT($factors$))

ORDER(VARS[bn]) $\rightarrow$ every ordering yields a valid algorithm
Try to minimize size of next factor to be constructed



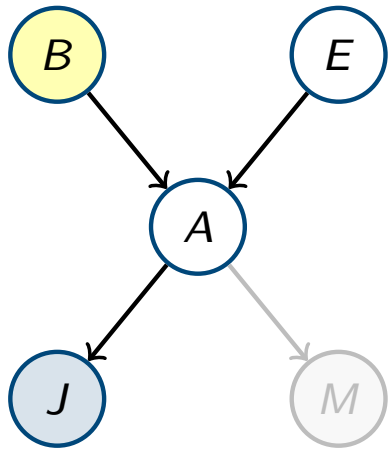
Irrelevant variables: can we skip some work?

A new query

$$P(JohnCalls \mid Burglary = true) = P(J \mid b).$$

- Query variable: J.
- Evidence: $B = true$.
- Hidden variables: E, A, M.

Mary's call is neither queried nor observed. Does its CPT need to enter the calculation?



Mary's contribution is always 1

For either value of A:

$$\sum_m P(m \mid A) = P(M = true \mid A) + P(M = false \mid A) = 1.$$

Why Mary's CPT disappears

Marginalizing the hidden variables gives:

$$P(J \mid b) = \sum_e P(e) \sum_a P(a \mid b, e) P(J \mid a) \underbrace{\sum_m P(m \mid a)}_{=1}.$$

Therefore:

$$\boxed{P(J \mid b) = \sum_e P(e) \sum_a P(a \mid b, e) P(J \mid a).}$$

Variable M and its local distribution $P(M \mid A)$ can be omitted when computing this query: we obtain exactly the same result.

Why does this work?

Mary is unobserved leaf node: no children, no other factor contains M. If her call were observed, we would select $P(m \mid A)$ instead of summing over. That factor would generally affect the result.

A graphical rule: retain the ancestral subgraph

Which nodes must we retain?

For a query $P(X \mid \mathbf{E} = \mathbf{e})$, retain:

- the query variable X;
- the evidence variables $\mathbf{E}$;
- all ancestors of the query and evidence variables.

All other nodes can be removed before inference.

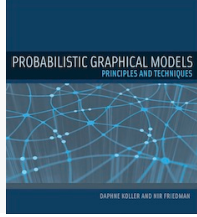
For $P(J \mid b)$:

$$\text{retained nodes} = \{J, B, A, E\}.$$

MaryCalls is outside this set, so it can be removed.

Why look at ancestors?

Repeatedly remove unobserved leaf nodes that are not queried. Each contributes a sum equal to 1, as Mary did.



Irrelevant variables: use d-separation

Predict whether the student gets a job, given the observed grade and SAT result:

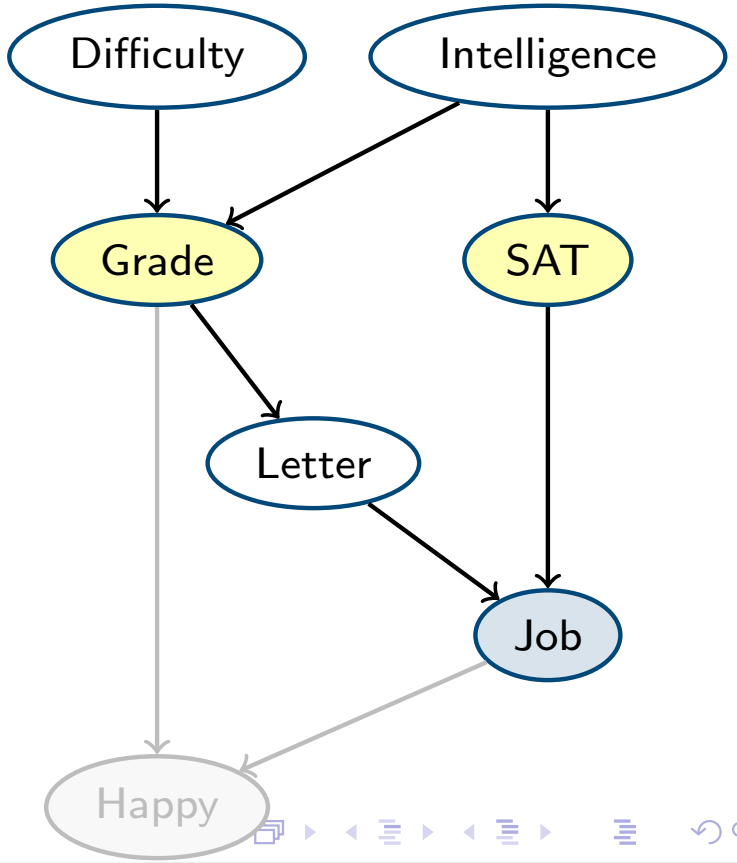
$$P(Job \mid Grade = g,\ SAT = s).$$

First, apply the ancestral-subgraph rule.

Happy is neither:

- query or evidence variable;
- ancestor of query or evidence variable.

It can therefore be removed.



Can we simplify further?

Difficulty and *Intelligence* are ancestors, but do we need their distributions for this query?

Observed variables block the remaining trails

After removing *Happy*, consider the trails to *Job*.

From *Difficulty*

$$Difficulty \rightarrow Grade \rightarrow Letter \rightarrow Job.$$

Grade is an observed non-collider: the trail is blocked.

From *Intelligence*

$$Intelligence \rightarrow Grade \rightarrow Letter \rightarrow Job,$$

$$Intelligence \rightarrow SAT \rightarrow Job.$$

The first trail is blocked by *Grade*; the second is blocked by *SAT*. Both are observed non-colliders.

Therefore:

$$\boxed{Job \perp \{Difficulty, Intelligence\} \mid Grade, SAT.}$$

What does this independence save us?

Theorem (Irrelevance by d-separation)

If Y is d-separated from the query variable X by the evidence variables $\mathbf{E}$, then Y is irrelevant as additional evidence for the query:

$$P(X \mid \mathbf{E} = \mathbf{e}, Y = y) = P(X \mid \mathbf{E} = \mathbf{e}),$$

whenever the conditioning event has positive probability.

The calculation reduces to

The remaining hidden variable needed for the query is *Letter*:

$$P(Job \mid g, s) = \sum_{l} P(Job \mid Letter = l, SAT = s)\, P(Letter = l \mid Grade = g).$$

Only the local models for *Letter* and *Job* are needed.

The simplification is specific to this query and its evidence.

What makes variable elimination expensive?

Variable elimination stores intermediate factors: much of the computational cost.

Factor size (for Boolean variables)

- a factor over 2 variables has $2^2 = 4$ entries;
- a factor over 10 variables has $2^{10} = 1024$ entries;
- a factor over 20 variables has 2^{20} entries.

A standard time bound is: $O(n\, d^k)$, where:

- n: number of variables;
- d: maximum number of values per variable;
- k: largest scope of a factor product formed before summing out a variable.

Why the elimination order matters

Different orders can create factors of different sizes. Smaller intermediate factors require less computation and memory.

How does network structure affect the cost?

Polytree

A Bayesian network is a polytree if any two nodes are connected by at most one path when arrow directions are ignored.

Polytrees

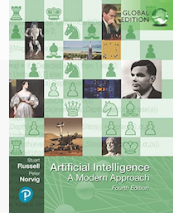
An appropriate elimination order keeps intermediate factors small relative to the local models. With bounded variable domains and bounded numbers of parents, exact inference can be performed in **time linear** in the number of nodes.

Theorem (General exact inference)

Exact probabilistic inference in unrestricted Bayesian networks is NP-hard.

Practical consequence

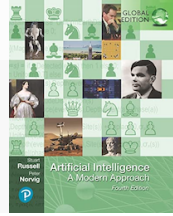
A compact network does not necessarily allow cheap inference. When intermediate factors become too large, we may need approximate inference.



Clustering algorithms

- Variable elimination OK for solving individual queries
- However, cost of estimating posterior probabilities for n variables in a polytree is $O(n^2)$
- **Clustering algorithms** (**join tree** algorithms) can do that in $O(n)$: used in commercial systems
- Idea is to join individual nodes to form cluster nodes (**meganodes**), obtaining a polytree
- Worst case still requires exponentially large CPTs
- Alternative approach: approximate methods

Exercise



Exam-style exercise: online shop

You are designing a new e-commerce system, to sell products online. You can display advertisements ("ads") of three types: book ads, toy ads, and holiday ads. However, your web site can only display one ad at a time. Your system should guess which type of ad has a higher chance of being clicked.

You don't have verified profile information on each customer. You can, however, guess at least some profile features, based on your domain knowledge and on your observations about which ads the customer does or doesn't click on ("clicking behaviour").

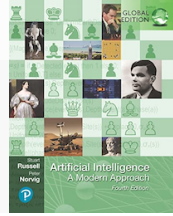
In particular, you have identified the following **profile features**:

- Young: is this a young customer (y, $\neg y$);
- Married: is the customer married (m, $\neg m$);
- Wealthy: is the customer wealthy (w, $\neg w$);
- Children: does the customer have children (c, $\neg c$).

The customer's **clicking behaviour** is instead determined as follows:

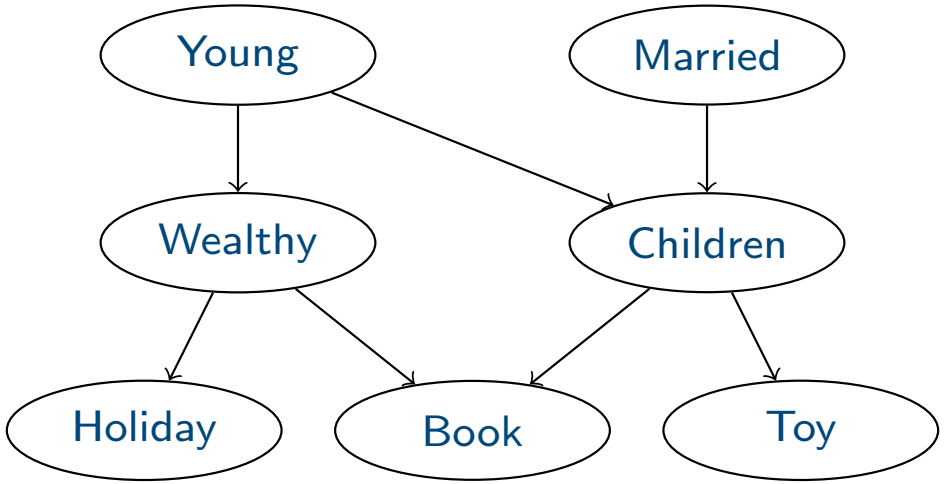
- Holiday: the customer clicks on a holiday ad (h, $\neg h$);
- Book: the customer clicks on a book ad (b, $\neg b$);
- Toy: the customer clicks on a toy ad (t, $\neg t$).

☞ How would you model this domain using a Bayes net?

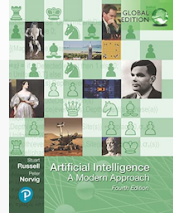


Exam-style exercise: online shop

Assume this is your network:

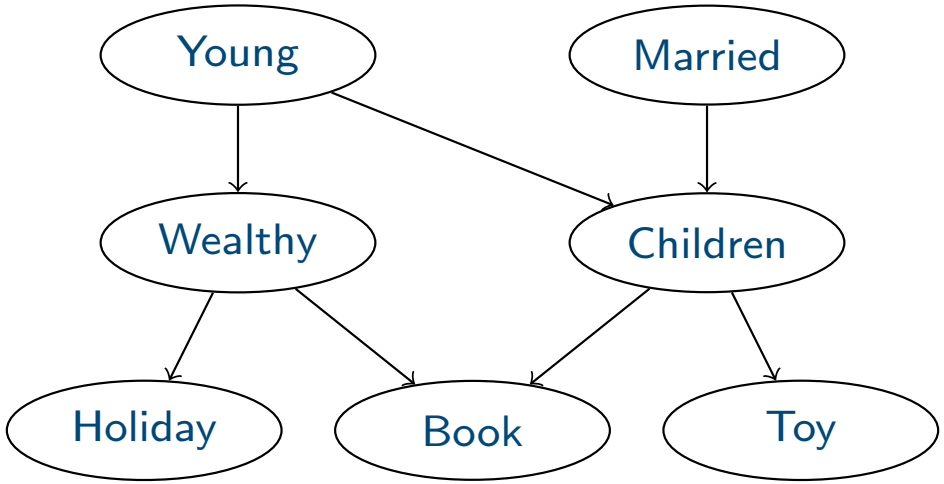


- Q1: *You first display a sequence of ads and record which types of ads get clicked; then, you use that data to infer some of the customer's profile features.*
  - ☞ *Which reasoning pattern are you following?*
  - ☞ *Show a query that uses the customer's observed clicking behaviour as evidence.*

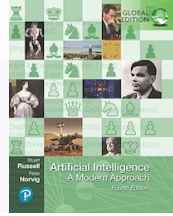


Exam-style exercise: online shop

Assume this is your network:

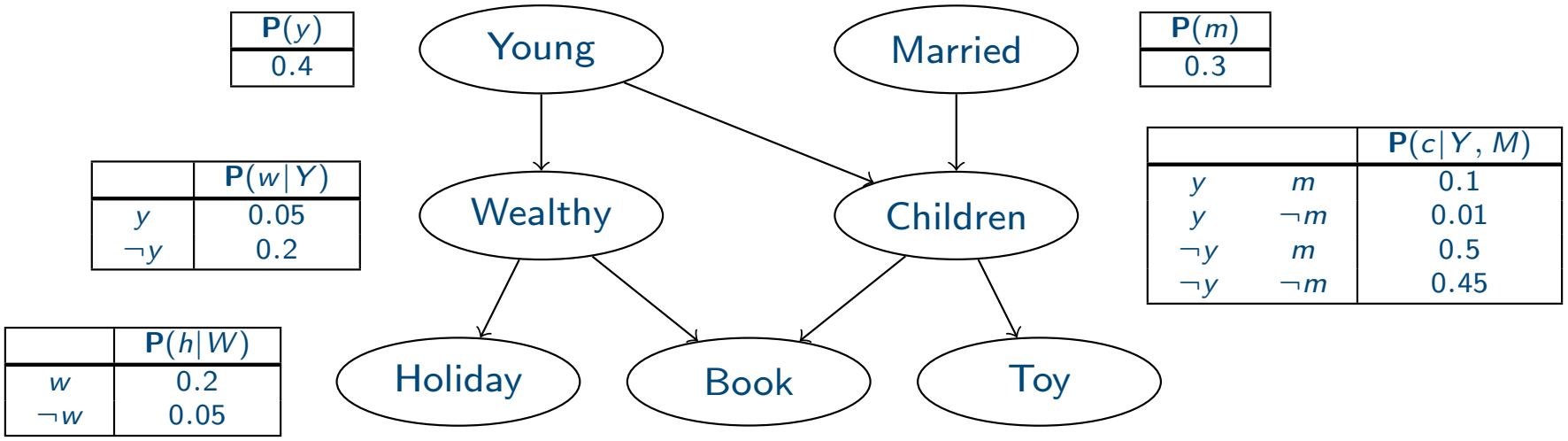


- Q2: *To fully define this Bayesian network, how many independent values are needed?*
- Q3: *Define a CPD for Children*



Exam-style exercise: online shop

Assume this is your network, with its parameters:



$\mathbf{P}(y)$
0.4

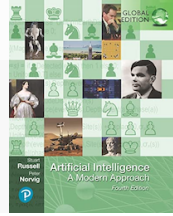
$\mathbf{P}(m)$
0.3

	$\mathbf{P}(w\|Y)$
y	0.05
$\neg y$	0.2

		$\mathbf{P}(c\|Y,M)$
y	m	0.1
y	$\neg m$	0.01
$\neg y$	m	0.5
$\neg y$	$\neg m$	0.45

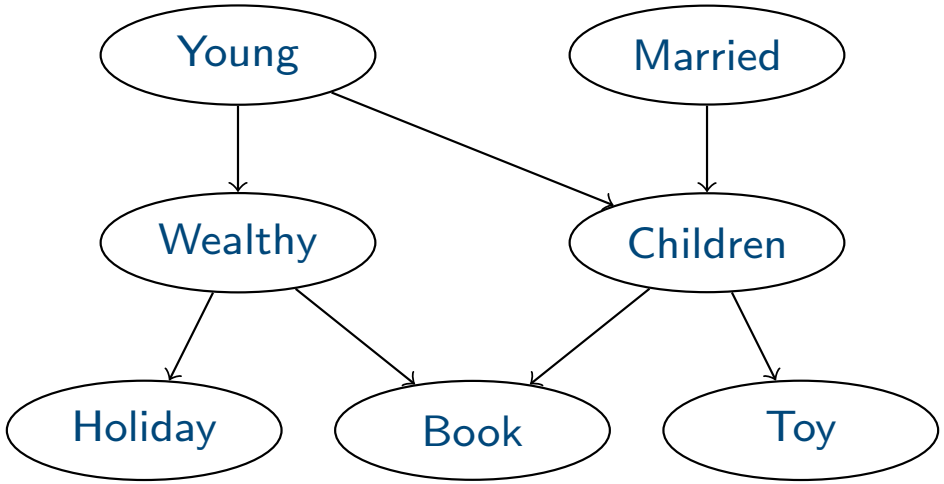
	$\mathbf{P}(h\|W)$
w	0.2
$\neg w$	0.05

- Q4: *Calculate $P(c|h)$ using variable elimination.*

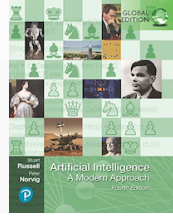


Exam-style exercise: online shop

Assume this is your network:

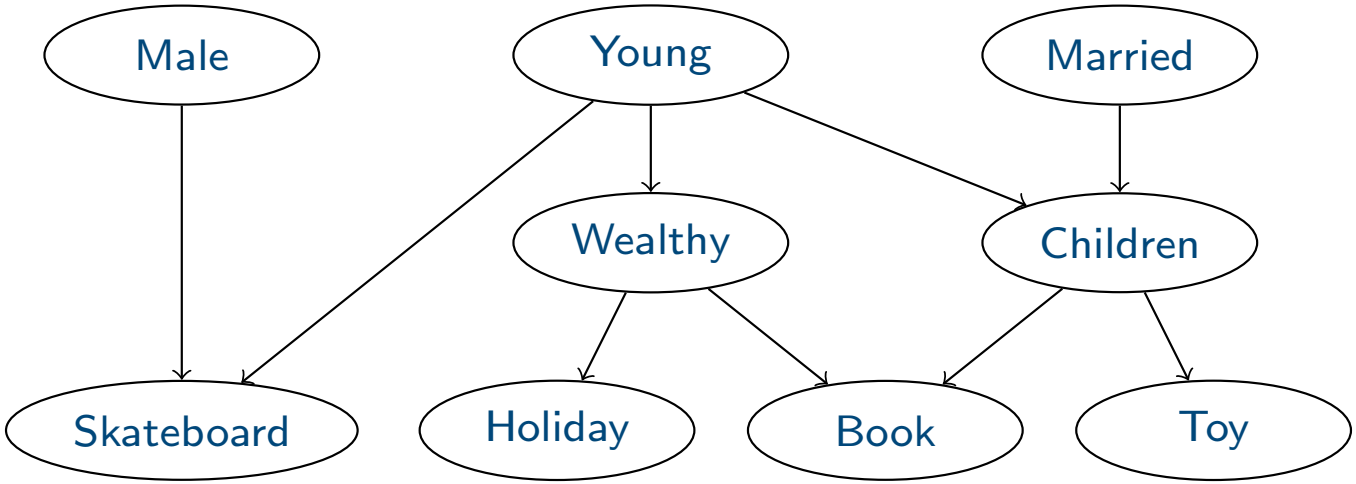


- Q5: *You expand your business to sell Skateboards: an article that's especially popular with young male customers.*
  - ☞ *How would you expand your Bayesian network to include also skateboard ads?*

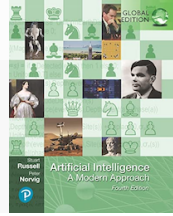


Exam-style exercise: online shop

Assume this is your extended network:

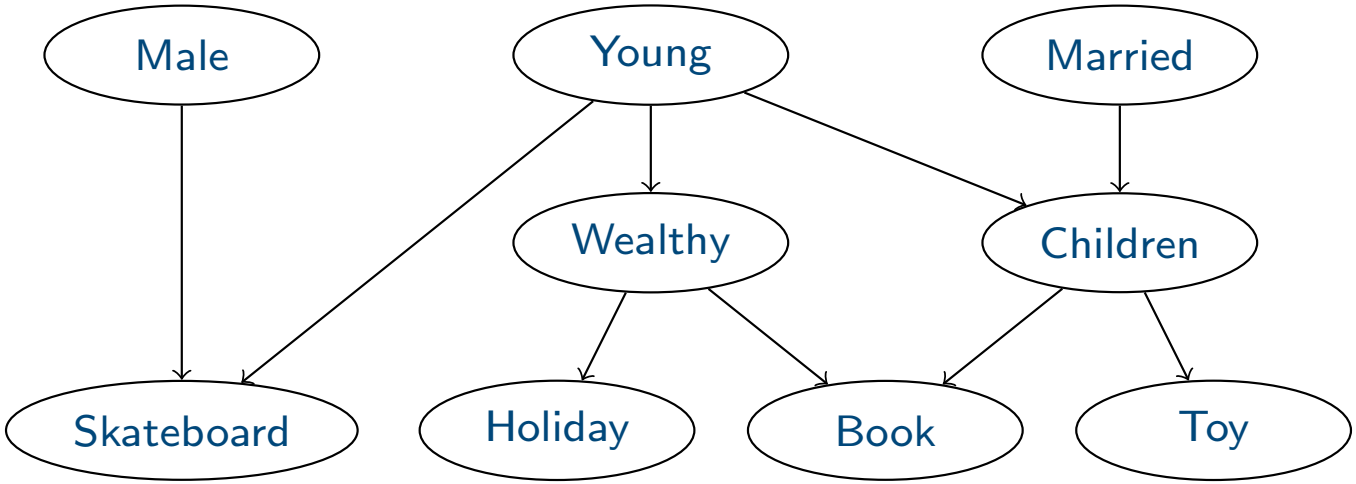


- Q6: *You display a skateboard ad, and observe that your customer doesn't click on it.*
  ☞ *Can this observation alone help predict if your customer is married?*



Exam-style exercise: online shop

Assume this is your extended network:



- Q7: *The same customer joins a fidelity programme and fills in a form where she declares to be a female parent. She chooses not to disclose any other information about herself.*
  - ☞ *Given this new evidence, does her not clicking on a skateboard ad give you any information about her being married?*

Suggested exercise from Russel & Norvig, 3rd Ed.

- 14.21 (soccer teams), (a)-(d)

Questions?

